PARA ACHAR == (CTRL + A & CTRL + H)

👾👾👾👾👾👾👾👾👾👾👾👾👾👾👾

AUTOMAÇÃO DE SHELL SCRIPTS EM PYTHON

F'STRING PARA SOMA DE NÚMEROS:

SCRIPT API PREÇO DE DÓLAR, EURO, IENE, LIBRA:

SCRIPT API CRIPTOMOEDAS:

IF AND ELIF AND ELSE:

1️⃣ **import = importa bibliotecas**

2️⃣ **VARIÁVEIS = PERGUNTAS**

3️⃣ **funções = cria o comando.**

4️⃣ **if, elif, elfe = decisão.**

5️⃣ **loop = repete código.**

MANUAL SINTAXE

👾👾👾👾👾👾👾👾👾👾👾👾👾👾👾

Como funciona a interação do python com a CPU?

Python não pode ser compilado para binário como C ou Rust, pois o Python é uma linguagem **interpretada**. Diferente do C ou Rust, onde o compilador traduz o código diretamente para a 'língua' da CPU (binário), o Python utiliza um intermediário chamado **Interpretador**, que no seu sistema fica localizado em `/usr/bin/python`."

O papel do `/usr/bin/python`

Pense nesse arquivo não como o seu script, mas como um **motor**. Ele é um programa robusto, escrito em C e já compilado, que sabe conversar com o Kernel do Linux e com a CPU. Quando você executa um script, você está, na verdade, entregando um texto para esse "motor" ler e executar as ordens em tempo real.

você pode ver o bytecode executando esse comando:
python -m dis soma.py

Python

Aqui irei anotar tudo que sei de python

1️⃣ import = importa bibliotecas.

1️⃣ Sistema / Linux / Arquivos

import os = Biblioteca do sistema operacional. mexe com arquivos, pastas e permissões.
{os.getcwd() = mostra pasta atual}
{os.listdir() = lista arquivos}
{os.mkdir() = cria pasta}
{os.path.expanduser("~") = transforma ~ em /home/user}

import subprocess = executa comandos Linux dentro do Python.
{subprocess.run(["ls"]) = executa comando}
{subprocess.run(..., capture_output=True) = captura saída}

import shutil = copiar mover apagar arquivos.
{shutil.copy() = copia arquivo}
{shutil.move() = move}
{shutil.rmtree() = apaga pasta inteira}

import pathlib = caminhos modernos de arquivos.
{Path("arquivo").exists() = verifica se existe}
{Path.home() = pasta home}

import glob = procurar arquivos usando padrão.
{glob.glob("*.txt") = lista txt}

import tempfile = cria arquivos temporários.
{TemporaryFile() = arquivo temporário}

import stat = permissões Linux.
{stat.S_IRWXU = permissões dono}

2 Entrada e Segurança

import getpass = esconde senha digitada.
{getpass.getpass()}

import argparse = argumentos no terminal.
{parser.add_argument()}

import sys = controla execução python.
{sys.exit() = encerra programa}
{sys.argv = argumentos CLI}

import logging = cria logs profissionais.
{logging.info()}
{logging.error()}

3 Datas e Tempo

import datetime = datas e horas.
{datetime.datetime.now()}
{datetime.timedelta()}

import time = controle tempo.
{time.sleep(5)}
{time.time()}

import calendar = calendários.
{calendar.month()}

4 Matemática

import math = matemática básica.
{math.sqrt()}
{math.pi}

import random = números aleatórios.
{random.randint()}
{random.choice()}

import statistics = média e estatística.
{statistics.mean()}

import decimal = cálculos precisos dinheiro.
{Decimal("10.50")}

5 Texto / Dados

import re = expressões regulares (buscar texto).
{re.search()}
{re.findall()}

import json = trabalhar JSON APIs.
{json.loads()}
{json.dumps()}

import csv = arquivos CSV.
{csv.reader()}

import pickle = salvar objetos python.
{pickle.dump()}
{pickle.load()}

import base64 = codificar dados.
{base64.b64encode()}

import hashlib = hashes criptografia.
{hashlib.sha256()}

6 Rede / Internet

import socket = comunicação rede.
{socket.socket()}

import http.client = HTTP manual.
{HTTPConnection()}

import urllib.request = baixar arquivos internet.
{urlretrieve()}

import requests = HTTP fácil APIs.
{requests.get()}
{requests.post()}

import aiohttp = HTTP assíncrono rápido.
{session.get()}

7 Multitarefa

import threading = executar várias tarefas.
{Thread()}

import multiprocessing = usar vários CPUs.
{Process()}

import asyncio = programação async moderna.
{async def}
{await}

8 Banco de Dados

import sqlite3 = banco local leve.
{connect()}
{cursor.execute()}

import psycopg2 = PostgreSQL.
{connect()}

import pymysql = MySQL.
{connect()}

9 Web / Automação

from flask import Flask = criar API web simples.
{@app.route()}

import django = framework web completo.
{models}
{views}

import fastapi = API rápida moderna.
{@app.get()}

import selenium = automação navegador.
{driver.get()}
{find_element()}

import beautifulsoup4 = ler HTML scraping.
{find()}
{select()}

🔟 Dados / IA
import numpy = matemática vetores rápida.
{np.array()}

import pandas = trabalhar tabelas dados.
{read_csv()}
{DataFrame()}

import matplotlib.pyplot = gráficos.
{plot()}
{show()}

import seaborn = gráficos estatísticos.
{heatmap()}

import scikit-learn = machine learning.
{fit()}
{predict()}

1️⃣0️⃣ Segurança / Pentest
import paramiko = SSH python.
{SSHClient()}

import scapy = manipular pacotes rede.
{sniff()}
{send()}

import cryptography = criptografia forte.
{Fernet()}

import nmap = usar nmap python.
{PortScanner()}

1️⃣2️⃣ Interface / Terminal
import curses = interface terminal Linux.
{addstr()}

import rich = terminal bonito cores.
{print()}
{progress}

import colorama = cores terminal Windows/Linux.
{Fore.RED}

1️⃣3️⃣ Outros MUITO usados

import collections = estruturas avançadas.
{Counter()}
{defaultdict()}

import itertools = loops avançados.
{product()}
{cycle()}

import functools = funções avançadas.
{lru_cache()}

import inspect = inspecionar código python.
{getsource()}

import traceback = mostrar erros completos.
{print_exc()}

🐧 **SSH / SERVIDORES / REMOTO**

import fabric = automação SSH profissional.

{Connection.run() = executa comandos remotos}

{put() = envia arquivos}

import asyncssh = SSH assíncrono rápido.
{connect()}
{run()}

import spur = executar comandos remotos/local fácil.
{LocalShell()}
{SshShell()}

import plumbum = shell scripting em python.
{local["ls"]()}
{ssh_machine()}

import invoke = automação tarefas estilo makefile.
{task()}
{run()}

🌐 REDE / PACOTES

import netifaces = informações interfaces rede.
{interfaces()}
{ifaddresses()}

import netifaces = informações interfaces rede.
{interfaces()}
{ifaddresses()}

import psutil = CPU RAM processos disco.
{cpu_percent()}
{process_iter()}
{disk_usage()}

import pyroute2 = manipular rede Linux.
{IPRoute()}
{link()}

import dpkt = analisar pacotes rede.
{ethernet.Ethernet()}

import pythonping = ping python.
{ping()}

🔎 SCANNER / SEGURANÇA

import impacket = protocolos rede SMB NTLM kerberos.
{SMBConnection()}

import pwntools = exploit development.
{process()}
{remote()}

import yara = detectar malware regras yara.
{compile()}
{match()}

import volatility3 = análise memória RAM.
{plugins}

import pyshark = wireshark python.
{FileCapture()}

📦 DOCKER / DEVOPS

import docker = controlar docker containers.
{containers.run()}
{images.pull()}

import kubernetes = controlar cluster kubernetes.
{client.CoreV1Api()}

import ansible_runner = rodar ansible python.
{run()}

import watchdog = detectar mudança arquivos.
{Observer()}

import pyinotify = monitorar arquivos Linux.
{add_watch()}

import rarfile = abrir rar.
{RarFile()}

import py7zr = arquivos 7zip.
{SevenZipFile()}

🔐 CRIPTOGRAFIA / HASH

import bcrypt = hash senha forte.

{hashpw()}

{checkpw()}

import passlib = gerenciamento senhas.
{verify()}

import pyjwt = tokens JWT APIs.
{encode()}
{decode()}

🌎 WEB / SCRAPING PESADO

import cloudscraper = bypass cloudflare scraping.
{get()}

import httpx = requests moderno rápido.
{get()}
{post()}

import playwright = automação navegador moderna.
{launch()}
{goto()}

📊 LOG / MONITORAMENTO

import prometheus_client = métricas servidores.

{start_http_server()}

import sentry_sdk = monitorar erros produção.
{capture_exception()}

import sentry_sdk = monitorar erros produção.
{capture_exception()}

import loguru = logs avançados fácil.
{logger.info()}
{logger.error()}

2️⃣ VARIÁVEIS = PERGUNTAS.

🧠 **VARIÁVEL PERGUNTANDO PARA O USUÁRIO**

ip = input("IP da máquina destino: ")

🔢 **VARIÁVEL COM NÚMEROS**

idade = 25

➕ **VARIÁVEL COM MATEMÁTICA**

resultado = 10 + 5

🔤 **VARIÁVEL TEXTO (STRING)**

nome = "matheus"

🧩 **VARIÁVEL COM F-STRING (JUNTAR TEXTO)**

ssh = f"ssh {usuario}@{ip}"

📋 **VARIÁVEL LISTA (VÁRIAS COISAS)**

tarefas = ["estudar", "treinar", "hack"]

📚 **VARIÁVEL DICIONÁRIO (CHAVE + VALOR)**

usuario = {

"nome":"matheus",
"id":1

}

pegar valor:

usuario["nome"]

🌐 VARIÁVEL API (JSON)

dados = requests.get(url).json()

pegar parte:

dados["ip"]

🎨 VARIÁVEL CORES TERMINAL

vermelho = "\033[31m"
reset = "\033[0m"

usar:
print(vermelho + "erro" + reset)
É dessa biblioteca:
from colorama import Fore

erro = Fore.RED

📁 VARIÁVEL CAMINHO ARQUIVO

home = os.path.expanduser("~")

🧾 VARIÁVEL ARQUIVO ABERTO

with open("teste.txt") as f:

f vira arquivo.

ler:

texto = f.read()

🔐 VARIÁVEL SENHA OU TOKEN API

api_key = "abc123"

ou

token = os.getenv("TOKEN")

🧪 **VARIÁVEL BOOLEAN (SIM OU NÃO)**

ativo = True

logado = False

🔄 **VARIÁVEL CONTADOR**

contador = 0

3️⃣ funções = cria o comando.

1️⃣ subprocess.run()

subprocess.run(["ls"])

= executa comando Linux.

{subprocess.run(["ls"]) = executa comando}
{capture_output=True = captura saída}
{text=True = transforma saída em texto}
{check=True = erro se comando falhar}

Ex:

```
resultado = subprocess.run(
["ls"],
capture_output=True,
text=True
)

print(resultado.stdout)
```

2️⃣ print()

```
print("ola")
```

3️⃣ input()

```
nome = input("Nome:")
```

= pergunta algo.

4 len()

len(lista)

= conta tamanho.

{len("abc") = 3}

5 type()

type(x)

= mostra tipo variável.

6 int()

idade = int("20")

= transforma texto em número.

7 str()

texto = str(10)

= transforma número em texto.

8 float()

altura = float("1.80")

= decimal.

9️⃣ **range()**

Loop.

range(10)

{range(5)=0 até 4}

🔟 **enumerate()**

Muito usada.

for i,t in enumerate(lista):

= número + item.

1️⃣1️⃣ **open()**

Abrir arquivo.

open("arquivo.txt")

{"r" = ler}

{"w" = apagar escrever}

{"a" = adicionar}

1️⃣2️⃣ **read()**

f.read()

= lê arquivo inteiro.

13 write()

f.write("texto")

= escreve arquivo.

14 close()

f.close()

= fecha arquivo.

(normalmente automático no with).

15 os.listdir()

os.listdir()

= lista arquivos pasta.

16 os.mkdir()

os.mkdir("teste")

= cria pasta.

17 os.remove()

os.remove("arquivo.txt")

= apaga arquivo.

18 os.path.exists()

os.path.exists("teste.txt")

= verifica se existe.

19 datetime.datetime.now()

datetime.datetime.now()

= data atual.

20 time.sleep()

time.sleep(5)

= pausa programa.

21 requests.get()

API.

requests.get(url)

= pega dados internet.

{.json() = transforma resposta JSON}

{.status_code = código resposta}

22 json.loads()

json.loads(texto)

= transforma texto JSON em variável.

23 json.dumps()

json.dumps(dados)

= transforma variável em JSON.

24 random.randint()

random.randint(1,10)

= número aleatório.

25 sorted()

sorted(lista)

= organiza lista.

26 sum()

sum(lista)

= soma tudo.

27 max()

max(lista)

= maior valor.

28 min()

min(lista)

= menor valor.

29 map()

Aplicar função.

map(int,lista)

= transforma vários itens.

30 lambda

Função pequena rápida.

lambda x: x+1

Ex:

sorted(lista,key=lambda x:x["idade"])

4 if, elif, else = decisão.

import subprocess

```
verificar_nmap = subprocess.run(
    ["which", "nmap"]
)


if verificar_nmap.returncode == 0:

    print("nmap já está instalado")


else:

    print("[+] instalando nmap")

    instalar = subprocess.run(
        ["sudo", "apt", "install", "-y", "nmap"]
    )

    if instalar.returncode == 0:

        print("[+] nmap instalado com sucesso")

    else:

        print("[+] erro ao instalar nmap")
```

5 loop = repete código.

```
import subprocess

while True:
```

```
teste = subprocess.run(["ping","-c","1","google.com"])

if teste.returncode == 0:

    print("Internet OK")

    break

print("Tentando novamente...")
```

MANUAL SINTAXE

SAIDA:

print("texto") = imprime texto
print(10) = imprime número
print("A", "B") = imprime múltiplos valores
print("A" + "B") = concatena strings
print(f"Nome: {nome}") = f-string (formatação moderna)
print("linha1\nlinha2") = quebra linha
print("col1\tcol2") = tab

ENTRADA:

input("Texto: ") = recebe texto (string)
int(input()) = converte entrada para inteiro
float(input()) = converte para decimal
str(valor) = converte para string

VARIÁVEIS:

x = 10 = inteiro
y = 3.14 = float
nome = "Matheus" = string
ativo = True = booleano
None = valor vazio
type(x) = mostra tipo

OPERADORES MATEMÁTICOS:

+ = soma
- = subtração
* = multiplicação
/ = divisão
// = divisão inteira
% = resto
** = potência

OPERADORES DE COMPARAÇÃO

== = igual
!= = diferente
> = maior
< = menor
>= = maior ou igual
<= = menor ou igual

OPERADORES LÓGICOS

and = E
or = OU
not = NÃO

IF | ELIF | ELSE

if condição: = inicia decisão
elif condição: = segunda condição
else: = caso contrário

Exemplo:

```
if idade > 18:
    print("Maior")
else:
    print("Menor")
```

LOOP FOR

for i in range(5): = repete 5 vezes
range(5) = 0 até 4
range(1,5) = 1 até 4
range(1,10,2) = passo 2
break = sai do loop
continue = pula repetição

LOOP LISTA

```
for item in lista:
    print(item)
```

LOOP WHILE

condição = 0
condição2 = 3
while condição < condição2: = repete enquanto verdadeiro
condição += 1
break = encerra loop
continue = pula iteração

LISTAS

lista = [1,2,3] = cria lista
lista[0] = acessa índice
len(lista) = tamanho
lista.append(4) = adiciona
lista.remove(2) = remove valor
lista.pop() = remove último
lista.insert(0, 10) = insere posição

TUPLAS

tupla = (1,2,3) = imutável

DICIONARIOS

d = {"nome":"Matheus"} = cria dict
d["nome"] = acessa valor
d.keys() = retorna chaves
d.values() = retorna valores
d.get("nome") = acessa seguro

CONJUNTOS

s = {1,2,3} = conjunto
s.add(4) = adiciona
s.remove(2) = remove

FUNÇÕES

def nome(): = define função
def soma(a,b): = função com parâmetro
return valor = retorna resultado

TRY / EXCEPT

try: = tenta executar
except: = captura erro
except ValueError: = captura erro específico
finally: = executa sempre

ARQUIVOS

open("a.txt","r") = ler
open("a.txt","w") = escrever
open("a.txt","a") = adicionar
with open("a.txt") as f: = contexto seguro
f.read() = lê tudo
f.readline() = lê linha
f.write("texto") = escreve

IMPORT

import os = importa módulo
import subprocess = executa comandos
from math import sqrt = importa função específica

SUBPROCESS

subprocess.run(["ls"]) = executa comando
resultado.returncode = código de saída
stdout=subprocess.DEVNULL = oculta saída

CONVERSÃO DE TIPOS

int() = converte para inteiro
float() = converte para decimal
str() = converte para string
bool() = converte para booleano

STRING METODOS

texto.lower() = minúsculo
texto.upper() = maiúsculo
texto.strip() = remove espaços
texto.split() = divide
texto.replace("a","b") = substitui

LIST COMPREHENSION

[x for x in range(5)] = cria lista
[x for x in lista if x > 2] = com condição

CLASSES (BASE)

class Pessoa: = define classe
def __init__(self): = construtor
self.nome = nome = atributo

DEBUG

print() = debug simples
type() = ver tipo
len() = tamanho

print() = debug simples
type() = ver tipo
len() = tamanho